

CONTRACT NOTE REWORK

DocuSign Integration

Estimate 2 - Technical Walkthrough

~7.1 developer days (4.1 required + testing)

Background

Once a contract note PDF has been produced, it still needs to be signed by the customer. Today that final step is manual: someone has to send the document out, chase the signature, collect the signed copy, and file it against the customer's record.

Estimate 2 automates that entire loop. When Estimate 1's render pipeline writes a finished PDF, this system automatically sends it to the customer for electronic signature via DocuSign, then retrieves the signed copy and attaches it to the customer's Salesforce record.

It is a headless, event-driven pipeline. There is no new screen in the Admin Portal, the whole thing runs automatically from the moment a PDF lands. The only visible outcome for the business is a signed contract appearing against the customer in Salesforce.

Builds directly on Estimate 1

This estimate is delivered inside `BrytBusinessServices`, the repository where Estimate 1's backend landed, and consumes the output of Estimate 1's render pipeline. That pipeline is a Step Functions state machine, so the trigger for this estimate is a new final step added to that state machine, right after the PDF is written. The step invokes the send process with the customer details already in hand, and has its own error handling so a signing hiccup never marks the render itself as failed or throws away the finished PDF.

Estimate 1 also needs a small, real change: today it produces the PDF but does not carry the customer's Salesforce reference through to the output. That reference has to be surfaced (alongside the offer reference and customer name) so this pipeline knows who to send each contract note to. That work is coordinated with the Estimate 1 pipeline owner.

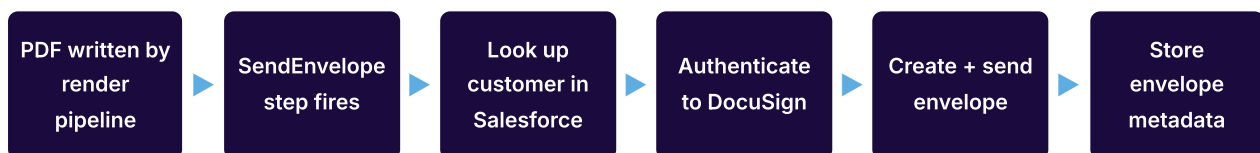
How it works

The system is two small Lambda functions with DocuSign and Salesforce either side of them.

The first Lambda runs when a PDF is produced: it looks up the customer in Salesforce, authenticates to DocuSign, and creates a signing envelope, which prompts DocuSign to email the customer. The second Lambda runs when the customer finishes signing: DocuSign calls back over a webhook, and the Lambda downloads the signed PDF, stores it, and attaches it to Salesforce.

In between those two moments, the customer signs at their own pace, so the flow is split into a send phase and a completion phase rather than one continuous run.

Send phase (automatic, on PDF creation)



DocuSign emails the signing request to the customer as soon as the envelope is sent.

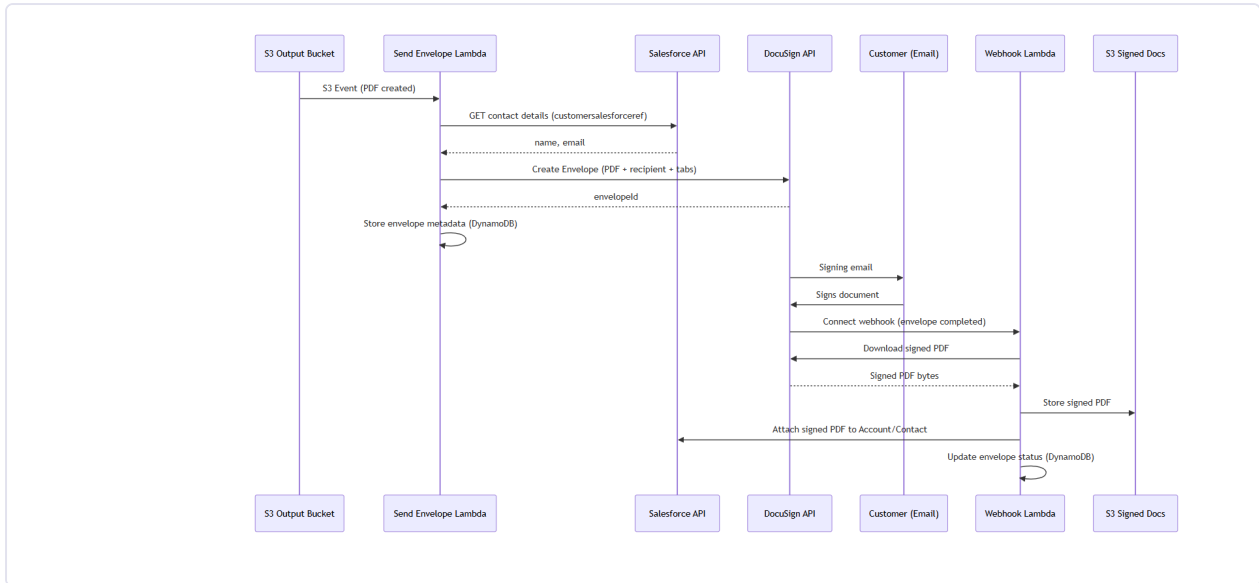
Completion phase (triggered when the customer signs)



A declined or expired envelope instead writes a notification record for operational follow-up.

End-to-end sequence

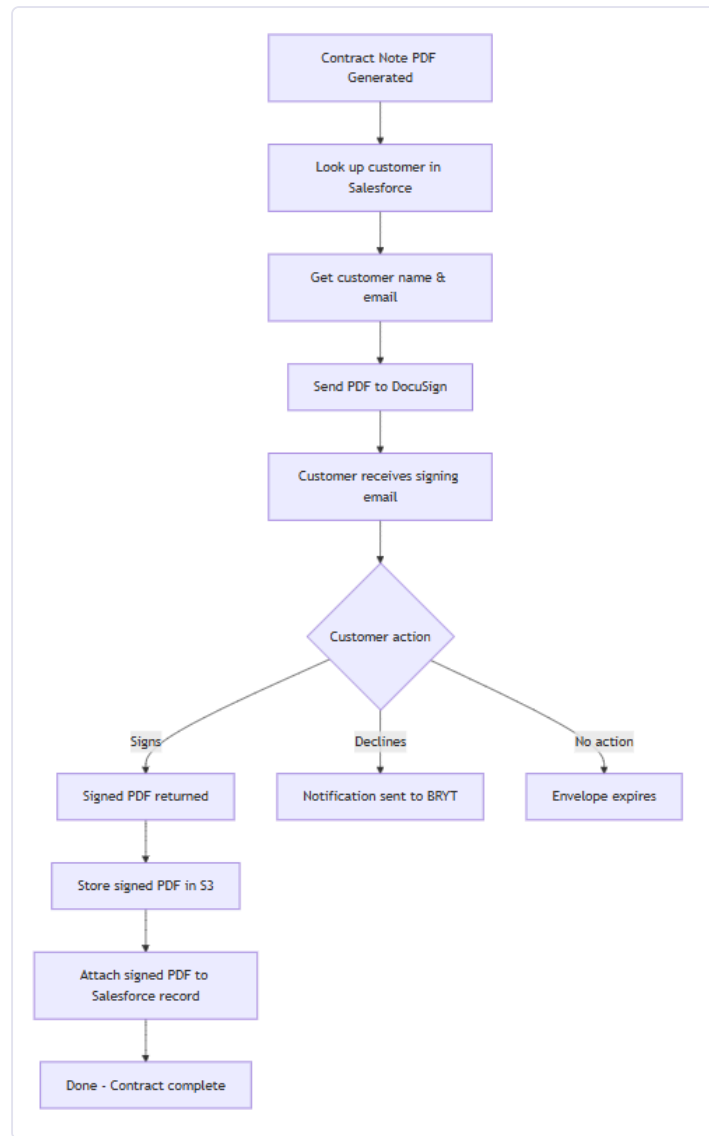
The full exchange between the render pipeline, the two Lambdas, DocuSign, the customer, and Salesforce. The top half is the send phase; the bottom half is the completion phase that runs after the customer signs.



Technical sequence diagram: PDF creation through to signed document stored in Salesforce.

Simplified flow

The same journey told for a non-technical audience, useful for explaining the process to stakeholders who care about the outcome rather than the mechanics.



Stakeholder-friendly view of the signing journey.

Key design decisions

A handful of choices keep the pipeline automated, secure, and consistent with existing BRYT patterns.

DECISION	CHOICE	WHY
DocuSign authentication	JWT Grant (server-to-server)	No human login needed; the token can be cached and refreshed for a fully automated pipeline
Trigger	New final step on the render state machine	Runs right after the PDF is written, with the customer details already in the payload; its own error handling keeps signing failures from failing the render
Status notifications	DocuSign Connect webhook (per-envelope)	Real-time updates instead of polling; per-envelope config avoids account-level admin setup
Signed document storage	S3 and Salesforce	S3 as the durable system of record; Salesforce for day-to-day business access
Salesforce integration	OAuth client, built fresh	No Salesforce API client exists in the landed code to reuse, so this is new (not a reuse) work
Resilience	Exponential backoff, 3 attempts	DocuSign and Salesforce calls can fail transiently; retries avoid losing a signed document
Webhook security	HMAC signature validation	The endpoint must be public for DocuSign to reach it, so every request is verified as genuine

The two Lambdas in detail

Send Envelope Lambda. Invoked by the render pipeline's final step with the customer's Salesforce reference already in the payload, it queries Salesforce for their name and email, authenticates to DocuSign, then creates an envelope containing the PDF with a signature field and the customer as the sole signer. Setting the envelope to sent makes DocuSign email the customer immediately. It records the envelope details in DynamoDB for traceability, and skips cleanly if an envelope already exists for that PDF so a retry can never double-send.

Webhook Lambda. Exposed via a public API Gateway endpoint, it receives DocuSign's callbacks. Every request is HMAC-validated first. On a completed event it downloads the signed PDF, stores it in S3, attaches it to the Salesforce record, and updates the status. On a declined or expired event it records the status and writes a notification for follow-up.

External integrations

The pipeline talks to two external services. Both authenticate with credentials held in AWS Secrets Manager.

SERVICE	WHAT WE CALL IT FOR	KEY OPERATIONS
DocuSign eSignature API	Sending documents for signature and retrieving signed copies	JWT auth; create + send envelope; download combined signed document
Salesforce REST API	Finding the customer and filing the signed contract	OAuth; query contact by reference; upload file (ContentVersion); link to record

What we track

A single DynamoDB table records one row per envelope, for debugging and traceability. There is no user-facing status screen in this phase.

FIELD	HOLDS
Envelope ID	DocuSign's identifier for the signing envelope (the primary key)
Salesforce reference	Links back to the customer record (also a lookup index)
Contract note S3 key	The original PDF that was sent
Customer name & email	Who the envelope was sent to
Status	sent, completed, declined, or expired
Signed PDF S3 key	Where the signed copy landed, once complete
Timestamps & error/decline reason	When it was created and last updated, plus any failure detail

Error handling and resilience

Because this pipeline produces legally significant documents, it fails safe. Any failure in the send phase (missing metadata, customer not found, no email on file, DocuSign or Salesforce errors) is logged to a shared error bucket and halts before an envelope is created, so nothing is half-sent.

In the completion phase, the two external calls that matter most, downloading the signed PDF and uploading it to Salesforce, each retry up to three times with exponential backoff. If they still fail, an error record is written for manual investigation, and the signed PDF is always safe in S3 regardless.

Open questions

This estimate carries assumptions that need BRYT confirmation. Each has a working assumption so delivery can proceed, but they should be resolved before or during implementation.

#	QUESTION	WORKING ASSUMPTION
1	Are there multi-party signing scenarios (BRYT rep, TPI), or always just the customer?	Single signatory (customer only). Multi-party would affect envelope routing and signing order.
2	Does BRYT have an existing DocuSign account?	Starting from scratch, new account, API credentials, and sandbox needed. No evidence found in AWS.
3	Is DocuSign's standard branded email acceptable, or is full control needed?	DocuSign sends the signing email directly, with BRYT branding configured in DocuSign settings.
4	Is an Admin Portal UI needed for envelope status tracking?	No UI this phase. Metadata is stored for debugging; the signed PDF in Salesforce is the visible outcome.
5	What Salesforce object does the customer reference map to, and is it queryable via the REST API?	Resolved at implementation time. Now more significant: with no existing Salesforce client, we confirm the object and that a REST/OAuth integration is available before build.
6	Are voided envelopes, resend, and reminders out of scope?	Out of scope. We handle completed, declined, and expired only. Retry/resend can come in a later phase.

Delivery breakdown

Estimate 2 is ~4.1 required days plus optional testing, ~7.1 days in total. Work is grouped as follows.

AREA	SCOPE
Infrastructure & utilities	Envelope metadata table + GSI, signed-docs S3 bucket, API Gateway webhook route, Secrets Manager, IAM, retry + error-record utilities (reusing Estimate 1's error bucket) as a CDK construct in BrytBusinessServices
Salesforce client (new)	OAuth auth, customer contact lookup, signed document upload (ContentVersion + link) — built fresh, no existing client to reuse

AREA	SCOPE
DocuSign client	JWT auth, envelope creation + send, signed document download, HMAC webhook validation
Metadata service	DynamoDB create / get / update / query-by-reference for envelope records
Send Envelope Lambda	Trigger handling, metadata extraction, and orchestration of the send phase
Webhook Lambda	HMAC validation, completion flow, declined/expired notification flow
Estimate 1 change	Surface the customer Salesforce reference through the render state machine to the PDF output, and add the trigger hook
Integration	CDK wiring into the contract-note stack, trigger wiring, end-to-end validation

Testing note

The build is designed around 10 correctness properties (trigger-to-envelope correlation, webhook validation, completion produces a signed PDF in both S3 and Salesforce, and fail-safe behaviour). Property-based and integration tests are marked optional in the plan and can be deferred for a faster MVP, but they are strongly recommended given the pipeline handles signed contracts.

CONTRACT NOTE REWORK

Training & Data Sources

Estimate 3 - Technical Walkthrough

~12.9 developer days (3a: 4 + 3b: 8.9)

Overview

Estimate 3 has two independent parts that can be prioritised separately. Part 3a is training and enablement, the documentation and materials that let the BRYT team run the new system themselves. Part 3b is data source extensibility, a software capability that lets business users enrich contract notes with data from new sources without developer involvement.

This document gives a brief summary of 3a, then focuses on the technical detail of 3b, which is the build effort.

3a. Training & Enablement (~4 days)

This part is a documentation and enablement effort rather than software. The goal is to get a small team of business users (roughly 2-5 people) productive on the template management system from Estimates 1 and 2 without ongoing developer support.

We intend to deliver a quick-start guide (zero to first template in under 30 minutes), a set of task-based how-to guides (create a template, update T&Cs, configure a rule, attach a data source), a data field reference cataloguing every field a designer can use, a one-page rules engine cheat sheet, template design-pattern guidance, and a troubleshooting FAQ. Optional screen recordings can follow once the UI is live.

These assets are best produced against the real UI, so drafts are written from the wireframes during the build and finalised with real screenshots afterwards, closing with a live training session and handover.

Why 3a comes after the build

Training materials lean on the actual screens. We draft them early from the mockups, then capture real screenshots and record walkthroughs once Estimates 1 and 3b are implemented, so the guides match what users actually see.

3b. Data Source Extensibility

Contract notes today can only show data that arrives in the contract payload. If the business wants to include something new, a credit score, a loyalty tier, a bespoke figure from another system, that has historically meant a developer change to the pipeline.

Estimate 3b removes that dependency. Business users manage data sources in SageMaker Unified Studio; once a source is subscribed to the contract note project, it becomes automatically discoverable in the Admin Portal. Users attach it to a template, use its fields in the section designer, and at render time the pipeline fetches the matching data and merges it in, all without a code change or redeployment.

The join key throughout is the BrytNumber (the customer reference already present in the contract payload). Every data source must expose a `bryt_number` column so it can be matched to the customer being rendered.

How it works

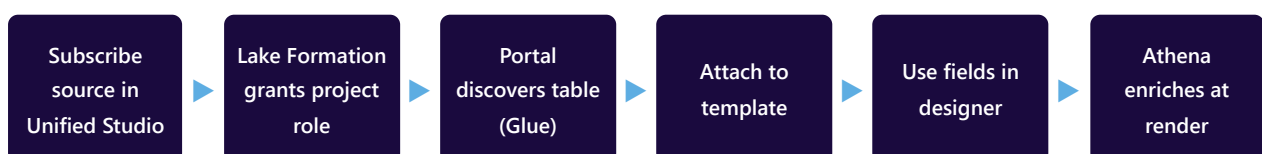
The capability spans three touch points: discovery, design, and render.

Discovery. When a user subscribes a data source in Unified Studio, Lake Formation grants land on the project role. The Admin Portal, assuming that same role, lists the newly available tables from the Glue Data Catalog, showing only those with a `bryt_number` column so they can actually be joined.

Design. A user attaches a data source to a template, and its columns then appear as fields in the section editor, grouped and namespaced by source, alongside the core contract fields. Shared sections automatically track which data sources they depend on.

Render. When the pipeline renders a template, it looks up the attached data sources, queries each one via Athena using the `BrytNumber`, and merges the results into the contract data before the sections are rendered.

From subscription to rendered field



A newly subscribed data source is usable end-to-end with no code change or redeployment.

The screen

Estimate 3b extends the existing Template Edit screen from Estimate 1 rather than adding new screens. A data sources panel is added, and the section editor gains a data source field group.

Template Edit - Data Sources panel

The screenshot shows the 'Edit Template: Pure Certainty HH' interface. It is divided into three main sections:

- Template Name:** A text box containing 'Pure Certainty HH'.
- Description:** A text box containing 'Fixed HH contract for commercial customers'.
- Sections (drag to reorder):** A list of five sections, each with a drag handle, a name, and buttons for 'shared', 'Edit in Designer', and 'Remove'. The sections are: 1. Header, 2. Contract Summary, 3. Pricing Table, 4. Credit Summary, and 5. Standard T&Cs v6.4. Below this list are two buttons: '+ New Section' and '+ Add Shared Section'.
- Data Sources:** A panel showing two attached data sources: 'credit_data' (3 cols) and 'consumption...' (5 cols). Each has a red 'X' button to detach it. Below these is a warning: 'Credit Summary' section requires credit_data. At the bottom of this panel is a '+ Attach Data Source' button.
- Available Fields Preview:** A box showing the fields available from the attached sources. It lists fields from 'credit_data' (credit_score, credit_grade, credit_limit) and 'consumption_forecast' (forecast_annual_kwh, forecast_peak_kw, forecast_method, forecast_date, confidence_pct).

A 'Save Changes' button is located in the top right corner of the interface.

The Template Edit screen gains a Data Sources panel showing which sources are attached to the template. From here a user attaches an available source or detaches one they no longer need.

KEY INTERACTIONS

- Attach Data Source opens a picker of available (unattached) sources
- Each attached source shows its name and column count
- Detach warns and lists affected sections if fields are in use
- Attached source columns become available in the section editor

BEHIND THE SCREEN

- Attachments are stored as records under the template in the existing DynamoDB table
- The picker is populated from the Glue catalog via the project role
- Only tables with a bryt_number column are offered
- Shared sections separately track their own data source dependencies

System architecture

The Admin Portal and render pipeline both reach data sources by assuming the Unified Studio project role, which already holds the Lake Formation grants. Discovery goes through the Glue Data Catalog; render-time enrichment goes through Athena. Attachment records live in the same DynamoDB table introduced in Estimate 1.



Access to data sources is inherited via the project role, so new subscriptions need no IAM changes.

Key design decisions

The design leans on the existing Unified Studio governance rather than reinventing access control.

DECISION	CHOICE	WHY
Data source access	Assume the Unified Studio project role	Inherits all Lake Formation grants automatically; no per-table IAM configuration
Catalogue discovery	Glue Data Catalog API	The standard AWS metadata layer; Unified Studio uses Glue underneath

DECISION	CHOICE	WHY
Render-time query	Athena	Serverless SQL over Glue/Iceberg tables; handles varied storage formats
Field namespacing	{source}.{column}	Avoids collisions between data sources and core contract fields; makes provenance clear
Template binding	Explicit attachment per template	Limits render-time queries to what's actually needed; keeps dependencies visible
Join constraint	Table must have a bryt_number column	Enforces join-ability and filters out tables that can't be matched to a customer
Section dependencies	Auto-tracked from field references	No manual config; derived by scanning which data source fields a section uses

Render-time enrichment

Enrichment slots into the existing render pipeline as a single new step between template selection and section rendering. Once a template is chosen, the pipeline gathers its attached data sources, reads the BrytNumber from the contract data, and queries each source in parallel via Athena. The results are merged into the contract data under a per-source namespace, then rendering proceeds exactly as before.

Enrichment step in the pipeline



Queries run in parallel to keep added latency low.

How enrichment handles the edge cases

SITUATION	BEHAVIOUR
Row found for the BrytNumber	Columns merged in under {table}.{column} and available to fields
No row for the BrytNumber	Warning logged; rendering continues with those fields empty
Multiple rows returned	First row used; a warning is logged
Athena query fails or times out	Error logged to the error bucket; rendering halts (no partial PDF)

What we store

No new table is needed. Two record types are added to the existing ContractNoteTemplates table.

RECORD	HOLDS
Template data source	The database + table attached to a template, display name, and who attached it / when
Shared section dependency	The data sources a shared section requires, derived from the fields it uses

RECORD	HOLDS
Field reference (in schema)	Data source fields are stored namespaced, e.g. credit_data.credit_score

Delivery breakdown (3b)

Estimate 3b is ~6.4 required days plus optional testing, ~8.9 days in total. Part 3a adds ~4 days of training/enablement effort.

AREA	SCOPE
Infrastructure & IAM	Project role trust policy for Lambda assumption, Athena workgroup + results bucket, DynamoDB record types
Glue discovery client	Assume project role, list tables, filter to bryt_number tables, fetch column detail
Data Source API	List available, get columns, attach, detach (with field-in-use check), list attached
Render enrichment	Athena query executor, enrichment orchestrator, integration into the render pipeline
Template Edit panel	Data sources panel, attach/detach with warnings, data source picker dialog
Section editor fields	Data source field group, dependency tracking, missing-dependency prompt, dependency display
Integration	CDK wiring for routes, role assumption, Athena, and end-to-end validation

A note on cost

Athena is billed by data scanned. For large data sources it is worth partitioning or pruning on bryt_number so each render-time lookup scans as little as possible. Queries also run in parallel across sources to keep the added render latency down.

Testing note

3b is designed around 10 correctness properties (bryt_number-only discovery, attachment round-trips, namespaced enrichment, graceful handling of missing rows, and fail-safe on query errors). Property-based and integration tests are optional in the plan and can be deferred for a faster MVP, but are recommended given enrichment feeds live customer documents.

CONTRACT NOTE REWORK

Bespoke Contracts

Estimate 4 - Technical Walkthrough

~7.8 developer days (4.8 required + testing)

Background

Not every customer fits the standard mould. Some need non-standard terms, a different pricing presentation, or VIP treatment, cases where the automated template-matching from Estimate 1 is not the right tool. Today these are handled by manually editing a PDF, which is slow, error-prone, and leaves no audit trail.

Estimate 4 gives business users a proper way to produce these one-off documents. A customer is flagged as bespoke on their Salesforce record; when their contract data arrives, the automated pipeline skips them and instead records that a bespoke contract is pending. A user then composes, renders, and sends that document manually through a dedicated area of the Admin Portal, using the same section editor they already know from Estimate 1.

Built almost entirely from reuse

This estimate is deliberately thin on new machinery. It reuses Estimate 1's section editor, shared sections, and render-and-stitch pipeline, and Estimate 2's DocuSign envelope logic. The recently delivered template-preview feature has already done much of the heavy lifting: the render pipeline can now be started on demand and polled for completion, the front end already has the poll-and-display logic, and the contract-note area is already access-gated. The new parts are the pipeline skip, the bespoke list and editor screens, extending section resolution to bespoke notes, and persisting render history.

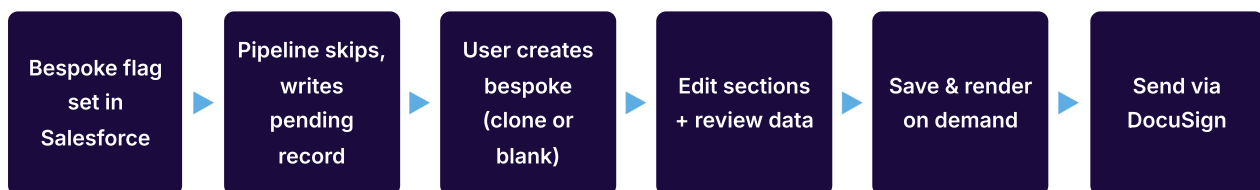
How it works

The journey starts in the automated pipeline. Before rendering, the pipeline checks the customer's bespoke flag in Salesforce. If it is set, the pipeline skips rendering entirely, produces no PDF, does not trigger DocuSign, and writes a pending record so the work surfaces in the Admin Portal.

From there it is a manual, user-driven flow. A user picks up the pending request and creates a bespoke contract note, either cloning an existing template as a starting point or beginning from a blank document. They compose it in the same section editor as standard templates, with the customer's actual contract data shown alongside for reference.

When ready, the user renders on demand to produce the PDF, reviewing and re-rendering as needed, with every render kept in history. Once happy, they send it for signature with a single button that reuses the Estimate 2 DocuSign flow.

End-to-end journey



The automated pipeline steps aside; a person drives the rest through the Admin Portal.

The skip is fail-safe

If Salesforce cannot be reached to check the flag, the pipeline proceeds with standard rendering rather than blocking. A missed bespoke flag is recoverable; a customer receiving no contract note at all is not.

Key design decisions

The decisions centre on isolation (bespoke edits must never touch standard templates) and reuse (don't rebuild what Estimates 1 and 2 already do).

DECISION	CHOICE	WHY
Bespoke flag source	A field on the Salesforce customer record	Single source of truth for customer data; no duplicated configuration
Cloned sections	Independent copies, not template references	Bespoke edits must not affect standard templates, and vice versa; full isolation
Render mechanism	Reuse the render state machine, started on demand and polled	The template-preview feature proved this pattern; same section-render-and-stitch logic, no duplicate rendering code
DocuSign	Reuse Estimate 2's envelope logic, triggered manually	Identical signing flow, just started by a button instead of automatically
Contract data	Stored alongside the bespoke record	Available both for the reference panel and for resolving fields at render time
Version history	Same pattern as standard sections (Estimate 1)	Consistent experience; reuses the existing version records

Screen-by-screen walkthrough

Two screens make up the bespoke experience. Both build on patterns from the Estimate 1 template editor.

1. Bespoke Contract Notes list

Bespoke Contract Notes						
Filter by status: All ▼						
Customer	BrytNumber	Offer Ref	Status	Modified	User	Actions
ACME INDUSTRIES LTD	BRYT004521	OP-58901	pending	2026-07-01	-	Create Bespoke
GOSPORT BOROUGH COUNCIL	BRYT002618	OP-56412	rendered	2026-06-28	Amina Ali	Edit PDF Send DocuSign
MERIDIAN HEALTHCARE	BRYT003890	OP-57234	draft	2026-06-25	Jodie Reaney	Edit
SOUTHAMPTON FC	BRYT001456	OP-55890	rendered	2026-06-20	Amina Ali	Edit PDF ✓ Signed
PINNACLE LOGISTICS	BRYT005102	OP-59012	failed	2026-06-18	Jodie Reaney	Edit

The entry point. A filterable table of every bespoke contract note and its status. Pending requests, customers who arrived via the pipeline skip, are highlighted so they get picked up.

KEY INTERACTIONS

- › Filter by status (pending, draft, rendered, failed)
- › Pending rows offer a Create Bespoke action
- › Edit opens the bespoke editor
- › Download PDF appears once a note has been rendered
- › Send via DocuSign appears on rendered notes

BEHIND THE SCREEN

- › Pending rows come from records the pipeline wrote when it skipped a customer
- › Each note shows customer, BrytNumber, offer reference, status, dates, and the user on it
- › Status reflects the latest render and DocuSign state
- › No per-user locking in this phase; any authorised user can edit any note

2. Bespoke Editor

Bespoke: GOSPORT BOROUGH COUNCIL

Save & Render Send via DocuSign

Customer: GOSPORT BOROUGH COUNCIL
BrryNumber: BRYT002648
Offer Reference: OP-5642
Status: ✓ Rendered (v2)
Cloned From: Pure Certainty HH
Created by: Amina Ali
Last modified: 2026-06-28 14:32

Render History

Version	Created By	Created At	Download PDF
v2	Amina Ali	2026-06-28 14:32	PDF
v1	Amina Ali	2026-06-28 14:32	PDF

✓ Last render successful - v2 - Jun 28 14:32 Download PDF

Sections (drag to reorder)

- 1 Header (shared v3) Edit History Remove
- 2 Custom Summary (v4) Edit History Remove
- 3 VIP Pricing (v2) Edit History Remove
- 4 Custom T&Cs (v1) Edit History Remove

+ New Section + Add Shared Section

Contract Data Reference

Search Fields...

Offer Details

offerid	4793
offername	GOSPORT BC - HH
producttype	Fixed
productname	Pure Certainty HH
contractstartdate	2025-09-09
contractenddate	2026-03-31

Customer

customername	GOSPORT BOROUGH
customerreference	BRYT002648
customerpostcode	PO2 5B
customercompany	Limited Company

Pricing

totalconsumptionmp	368.74
numberofmpans	1
paymenttermdays	28

MPANs

mpan	2000058045455
meterid	HH

Where the document is composed. The section management is the same as the Estimate 1 template editor, with two additions: a contract data reference panel and a render history panel.

KEY INTERACTIONS

- Add, remove, reorder, and edit sections in the pdf-me designer
- Add shared sections (headers, footers, T&Cs) from the existing library
- The reference panel shows the customer's actual field values while editing
- Save & Render produces the PDF on demand; re-render after edits
- Send via DocuSign appears once rendered

BEHIND THE SCREEN

- Sections live under the bespoke note, as independent copies if cloned
- The reference panel reads the stored contract JSON, grouped and searchable
- Each render appends a version to the render history (append-only)
- Section edits carry version history, same as standard templates

On-demand rendering and history

Unlike the automated pipeline, bespoke notes render when the user asks. Save & Render resolves the note's sections, then starts the same render-and-stitch state machine used by the standard pipeline (the very mechanism the template-preview feature already exercises) using the customer's stored contract data. The front end polls for completion and then writes the resulting PDF to S3. The only pipeline change is teaching the render's first step to resolve a bespoke note's own section copies, rather than only a published template's sections.

Every render is preserved. Each one appends a new version to the render history, with the timestamp, the user who triggered it, and a link to that PDF, so a user can always retrieve an earlier version. The current version is clearly marked, and it is the one used when the note is sent for signature.

Status lifecycle

A bespoke note moves through a small set of states as it is worked on.

STATUS	MEANING
pending	The pipeline skipped this customer; awaiting someone to pick it up
draft	A bespoke note has been created and is being composed
rendering	A render is in progress
rendered	A PDF has been produced and is ready to download or send
failed	The last render failed; the error is recorded for the user to see

Sending for signature

Once a note is rendered, a Send via DocuSign button appears. It reuses Estimate 2's flow wholesale: it looks up the customer's contact details in Salesforce, creates a DocuSign envelope around the latest rendered PDF, and stores the envelope so its status (sent, completed, declined, expired) shows against the note. Completion is handled by the same Estimate 2 webhook, so a signed bespoke contract lands in S3 and Salesforce exactly like a standard one.

What we store

Bespoke records follow the same single-table pattern as Estimate 1, keyed under the bespoke note rather than a template.

RECORD	HOLDS
Pending bespoke	Written by the pipeline skip: BrytNumber, offer reference, customer name, contract data location
Bespoke contract note	The note's metadata: customer, status, current render, DocuSign envelope + status, who created/updated it
Bespoke section	Each section's name, order, shared reference, schema location, and version, independent of any template
Render history	One row per render: version, PDF location, timestamp, user, success/failure

Delivery breakdown

Estimate 4 is ~4.8 required days plus optional testing, ~7.8 days in total. Much of the effort is wiring together existing capabilities.

AREA	SCOPE
Pipeline skip	Salesforce bespoke-flag check (fail-safe), pending record creation, contract data stored for later
Bespoke API	List, create (clone or blank), get, update, delete, section management, contract-data endpoint
On-demand render	Extend section resolution to bespoke notes, start/poll render handlers (reusing the template-preview pattern), render history, download links
Manual DocuSign	Send handler reusing Estimate 2 envelope logic; envelope status on the note
Bespoke module (Angular)	List screen, editor, contract-data reference panel, render history, creation/clone dialog; reuses the existing group guard and render poll/display logic
Integration	CDK routes, IAM (start/describe the render state machine), portal navigation, end-to-end validation

Testing note

The build is designed around 7 correctness properties (no automated output for flagged customers, pending-record creation, clone isolation, valid on-demand render, append-only history, and correct-PDF signing). Property-based and integration tests are optional in the plan and can be deferred for a faster MVP, but are recommended given bespoke notes are customer-facing contracts.

CONTRACT NOTE REWORK

Comparison Audit

Estimate 5 - Technical Walkthrough

~12.4 developer days (incl. prompt iteration)

Background

BRYT suspects that some contract note PDFs may be edited by hand after they are rendered but before they reach the customer, sidestepping the controlled template system. Because the edit happens outside any system, there is currently no way to know whether it is occurring or how often.

Estimate 5 is an audit tool that answers that question. For a batch of contract notes it retrieves the original rendered PDF and the version that was actually emailed to the customer, compares the two with AWS Bedrock, and reports any differences it finds.

This is a developer-operated analytical tool, not a customer- or business-facing product. It runs ad-hoc (for example monthly) over a list of references and produces results that are queried with Athena and delivered to BRYT as a spreadsheet.

How it differs from the other estimates

Estimates 1-4 build the production system. Estimate 5 is a separate investigative tool that runs alongside it. There is no UI and no live integration, just a batch pipeline a developer triggers when an audit is needed.

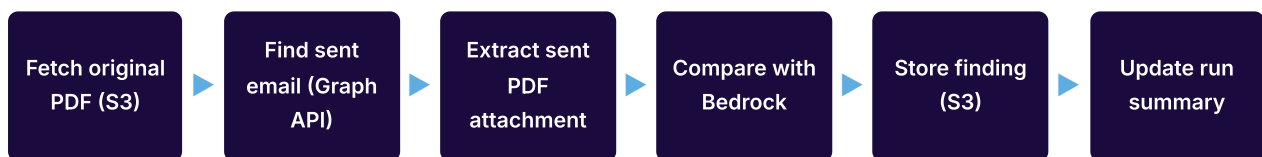
How it works

A developer supplies a list of contract note references (up to 500 per run). A Step Function iterates the list, and for each reference it does three things: finds the original rendered PDF in S3, finds the sent PDF by searching the Outlook mailbox via the Microsoft Graph API, and, if both are available, asks Bedrock to compare them.

The comparison result, identical or not, and if not, what changed, is written to S3 as a finding. A run-level summary tracks how many matched, mismatched, or errored. Once the run finishes, the findings are queried with Athena and turned into a spreadsheet for BRYT.

The two documents come from deliberately different places: the original from the system's own S3 output, and the sent version from the actual email, because the email is the source of truth for what the customer really received.

Per-record comparison flow



If either PDF is missing, an error finding is recorded and the run continues to the next record.

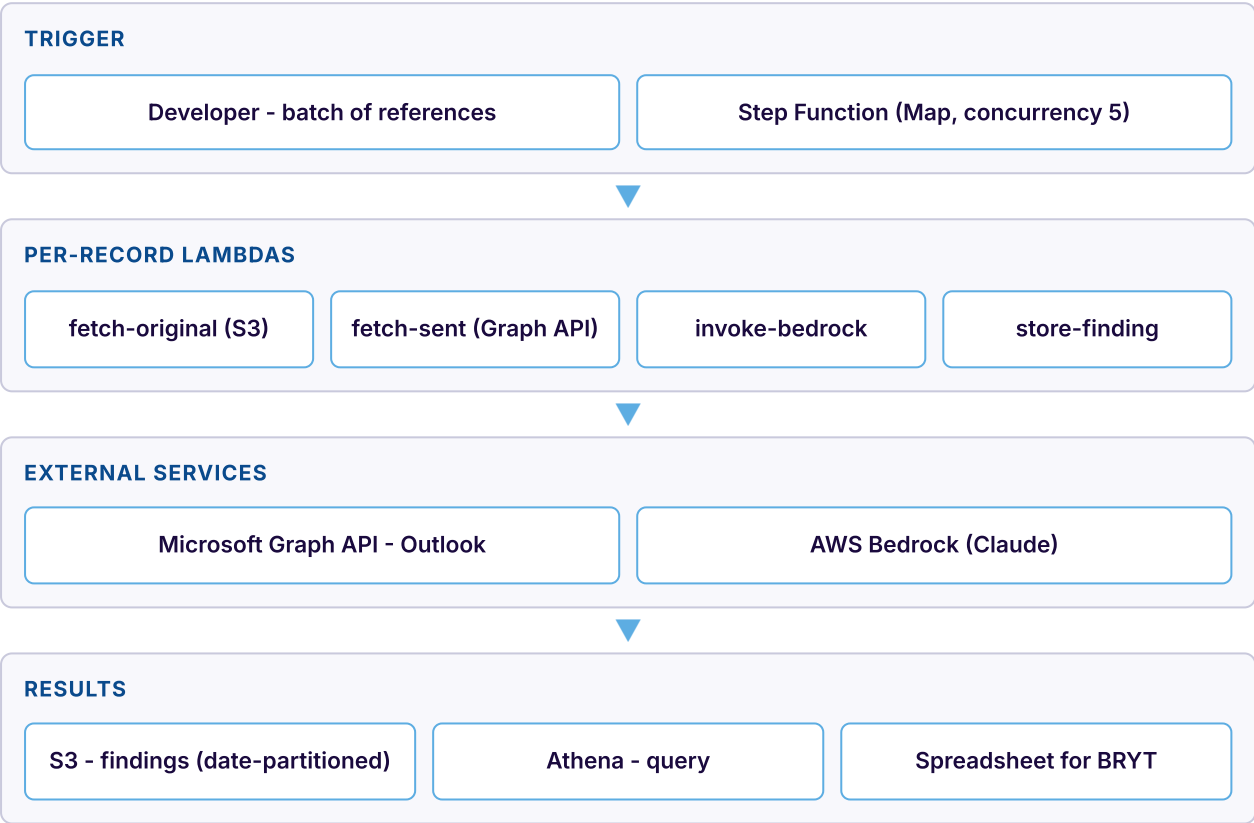
Run lifecycle



Concurrency is capped at 5 to stay within Graph API rate limits.

System architecture

A Step Function orchestrates a handful of small Lambdas, each responsible for one step. The two PDFs are sourced from S3 and Outlook respectively, compared by Bedrock, and the findings land back in S3 where Athena can query them.



A developer-operated batch tool; nothing here runs in the live contract note path.

Key design decisions

The design favours proven, low-overhead building blocks that suit an ad-hoc analytical tool.

DECISION	CHOICE	WHY
Orchestration	Step Functions with a Map state	Built-in iteration, retries, and error handling; a proven pattern
Original PDF source	S3 (existing output bucket)	The rendered original is already stored there by the current system
Sent PDF source	Microsoft Graph API (Outlook search)	The sent email is the source of truth for what the customer actually received

DECISION	CHOICE	WHY
Comparison engine	AWS Bedrock (Claude)	Compares documents textually and visually, and can describe nuanced differences
Results storage	S3 JSON, date-partitioned	Directly queryable via Athena; no database or ETL needed
Prompt management	SSM Parameter Store, versioned	The comparison prompt can be refined without redeploying; each finding records the version used
Operating model	Developer-operated, ad-hoc batches	Not a production system; monthly runs with manual analysis and reporting

The comparison itself

For each contract note where both PDFs are in hand, Bedrock is given the two documents and a prompt asking whether they are identical and, if not, to describe each difference with its location and severity. The response is stored as a structured finding: a match/mismatch verdict, a list of differences, and a one-line summary.

Getting this right is mostly a prompt-engineering exercise. The prompt lives in Parameter Store rather than in code so it can be tuned quickly, and each finding records which prompt version produced it. The estimate explicitly budgets for several iteration cycles, running a batch, reviewing the signal-to-noise with BRYT, and refining, until the output reliably flags meaningful edits without drowning them in noise.

What a finding records

Each finding is one JSON record per contract note, written to S3 and queryable in Athena.

FIELD	HOLDS
Offer reference / document ID	Which contract note this finding is for
Run ID & timestamp	Which batch produced it and when
Status	match, mismatch, or error
Original & sent PDF keys	Where each source document was found (sent key only if located)
Comparison result	Bedrock's verdict: identical flag, list of differences, summary
Prompt version	Which prompt version produced the result (for traceability)
Error message	Populated when a step could not complete

How the edge cases are handled

SITUATION	BEHAVIOUR
Original PDF not in S3	Record 'error: original not found'; continue to next record
No matching sent email	Record 'error: email not found'; continue

SITUATION	BEHAVIOUR
Multiple matching emails	Use the most recent; log a warning
Email has no PDF attachment	Record an error finding; continue
Bedrock call fails	Retry once, then record 'error: comparison failed'; continue
Bedrock reply unparseable	Store the raw response and flag it for manual review

Open questions

This estimate has a hard external dependency on Microsoft 365 access. These need BRYT confirmation before work can complete.

#	QUESTION	WHY IT MATTERS
9	Can BRYT provide an Azure AD app registration with Mail.Read (or Mail.ReadBasic) for the Graph API?	This is the critical dependency, without it the sent PDFs cannot be retrieved and the tool cannot function.
10	Which mailbox(es) are contract notes sent from?	Determines which mailbox the search runs against, could be one shared mailbox or several users.
11	How do we correlate a sent email to a specific contract note?	Assumption: the subject line or attachment filename contains the offer reference (e.g. OP-56412). Needs confirmation.

Critical dependency

Question 9 is a blocker rather than a detail. The whole tool depends on read access to the mailbox where contract notes are sent, which requires BRYT's M365 admin to grant the permission. It is worth securing this early, as everything else is downstream of it.

Delivery breakdown

Estimate 5 is ~12.4 days. Unlike the others, a large share is prompt iteration and analysis rather than pure build.

AREA	SCOPE
Infrastructure	Step Function, the six Lambdas, results S3 bucket, Secrets Manager, SSM prompt parameter, IAM, Athena/Glue table
Graph API integration	OAuth client-credentials auth, mailbox search, PDF attachment extraction
Bedrock integration	Initial comparison prompt, model invocation, structured response parsing

AREA	SCOPE
Prompt iteration	3-5 cycles of running batches, reviewing output with BRYT, and refining the prompt
Analysis & reporting	Athena queries for common patterns, producing the spreadsheet report for BRYT
Buffer	Graph API limitations and Bedrock output tuning

A note on effort

This estimate is less predictable than the others because its value depends on comparison quality, which is reached through iteration rather than a fixed build. The figure includes that iteration and a buffer for the Graph API and Bedrock tuning; the underlying spec notes a wider 13-20 day range depending on how many refinement cycles BRYT wants.